

PRÁCTICAS PROFESIONALIZANTES 2026

Escuela de Artes Visuales Martín A. Malharro

INFORME TÉCNICO — EQUIPO A

Documentación Técnica y Arquitectura

Integrantes:

Hormahechea Ignacio

Segura Luca

Perez Juan

Villamayor Francisco

Docente a cargo: Tresser Lucas

Mar del Plata, Buenos Aires, Argentina — 2026

Etapa 1 — Relevamiento y Análisis del Sistema

La Etapa 1 corresponde al relevamiento inicial y documentación del código fuente del sistema web de la Escuela de Artes Visuales Martín A. Malharro. Se realizó la documentación del código relacionado con Next.js y React. Se revisaron y documentaron 25 archivos de código, agregando comentarios explicativos sobre la función de cada archivo, el funcionamiento de los componentes y la lógica implementada en cada una de las funciones.

Archivos Documentados

Los siguientes 25 archivos fueron analizados y documentados durante la Etapa 1:

- config.js
- layout.js
- page.js
- useUser.js
- login/page.js
- registrar/page.js
- auth.js
- iniciarSesion.js
- loginWithGoogle.js
- registrar.js
- checkRole.js
- handleSave.js
- agenda.js
- carrusel.js
- document.js
- userMenu.js
- usina.js
- imagen.js
- imagenById.js
- text.js
- textoById.js
- acordeon.js
- acordeonById.js
- crearUsina.js
- usinaProtegida.js

Observaciones Detectadas durante el Relevamiento

Durante el proceso de revisión se detectaron las siguientes observaciones técnicas relevantes:

1. En auth.js, el token se almacena utilizando la clave "token", mientras que el resto del sistema lo busca mediante "jwt", lo que provoca que las funciones relacionadas no tengan un efecto real dentro del funcionamiento general.
2. La protección del formulario de Usina únicamente verifica la existencia de un token en el navegador, sin implementar una validación desde el lado del servidor.
3. El token administrativo de Strapi se encuentra expuesto como una variable pública, lo que permite su visualización desde el navegador.

Diagramas de Arquitectura

Explicación de la arquitectura:

- El usuario accede al sistema desde un navegador web.
- El frontend desarrollado con Next.js se encarga de mostrar la interfaz y manejar la navegación.
- El frontend realiza solicitudes a la API o CMS para obtener información (como noticias, agenda o contenido).
- Google OAuth se utiliza para la autenticación de usuarios (inicio de sesión con cuenta de Google).

Estructura de Carpetas del Proyecto

A continuación se detalla la estructura completa del repositorio:

```
malharro/
├── docs/
│   ├── ajustes_webmalharro.pdf
│   ├── informe_funcionamiento.pdf
│   └── malharro.pdf
├── malharroweb_src/
│   ├── css/
│   ├── images/
│   ├── js/
│   └── index.html
├── public/
│   ├── img/
│   ├── file.svg
│   ├── globe.svg
│   ├── next.svg
│   ├── vercel.svg
│   └── window.svg
├── src/
│   ├── app/
│   │   ├── componentes/
│   │   │   ├── basicos/
│   │   │   │   ├── acordeon/
│   │   │   │   │   ├── acordeon.js
│   │   │   │   │   └── acordeonbyid.js
│   │   │   │   ├── imagen/
│   │   │   │   │   ├── imagen.js
│   │   │   │   │   └── imagenbyid.js
│   │   │   │   └── texto/
│   │   │   │       ├── text.js
│   │   │   │       └── textobyid.js
│   │   └── agenda.js
```

```
├── carrusel.js
├── document.js
├── usermenu.js
├── usina.js
├── crearComponentes/
│   ├── crearusina.js
│   └── usinaprotegida.js
├── login/
│   ├── auth.js
│   ├── iniciarsesion.js
│   ├── loginwithgoogle.js
│   └── registrar.js
├── validacion/
│   ├── checkrole.js
│   └── handlesave.js
├── hooks/
│   └── useuser.js
├── login/
│   └── page.js
├── registrar/
│   └── page.js
├── config.js
├── favicon.ico
├── layout.js
├── page.js
├── page.module.css
├── styles/
│   ├── componentes-styles.css
│   └── styles.css
├── .gitignore
├── documentacion.md
├── jsconfig.json
├── next.config.mjs
├── package-lock.json
├── package.json
└── readme.md
```

Explicación de Carpetas y Archivos

docs/: contiene manuales, informes y documentación del proyecto.

malharroweb_src/: recursos y archivos heredados de la versión anterior de la web.

- css/: hojas de estilo antiguas.
- images/: imágenes y recursos gráficos.

- `js/`: scripts de soporte antiguos.
- `index.html`: página principal de la versión anterior.

`public/`: archivos públicos accesibles desde el navegador.

- `img/`: carpeta para imágenes públicas.
- `svg`: iconos y recursos gráficos del proyecto.

`src/`: contiene el código fuente principal de la aplicación.

- `app/`: núcleo principal basado en `next.js`.
- `componentes/`: componentes reutilizables de la interfaz.
- `basicos/`: componentes generales utilizados en varias secciones.
- `acordeon/`: lógica para contenido desplegable.
- `imagen/`: manejo y visualización de imágenes.
- `texto/`: manejo y formato de textos.
- `agenda.js`: visualización de eventos y actividades.
- `carrusel.js`: componente de slider o carrusel.
- `document.js`: manejo de documentos o contenido.
- `usermenu.js`: menú interactivo del usuario.
- `usina.js`: componente relacionado con publicaciones o proyectos.
- `crearComponentes/`: herramientas para generar contenido dinámicamente.
- `crearusina.js`: creación de contenido usina.
- `usinaprotegida.js`: contenido protegido con validación.
- `login/`: lógica de autenticación de usuarios.
- `auth.js`: control de autenticación.
- `iniciarsesion.js`: formulario de inicio de sesión.
- `loginwithgoogle.js`: acceso mediante google oauth.
- `registrar.js`: registro de nuevos usuarios.
- `validacion/`: control de permisos y validaciones.
- `checkrole.js`: verificación de roles de usuario.
- `handlesave.js`: control y validación de guardado.
- `hooks/`: funciones reutilizables de lógica personalizada.
- `useuser.js`: manejo del estado y datos del usuario.
- `login/`: página de inicio de sesión.
- `registrar/`: página de registro de usuarios.
- `config.js`: configuración global del sistema.
- `favicon.ico`: ícono de la pestaña del navegador.
- `layout.js`: estructura base compartida entre páginas.
- `page.js`: página principal del sitio.
- `page.module.css`: estilos específicos de la página principal.
- `styles/`: hojas de estilo generales.
- `componentes-styles.css`: estilos para componentes.
- `styles.css`: estilos generales de la aplicación.

`.gitignore`: define archivos y carpetas que no deben subirse al repositorio.

`documentacion.md`: documentación técnica del proyecto.

`jsconfig.json`: configuración de rutas y soporte para el editor.

`next.config.mjs`: configuraciones técnicas de `next.js`.

`package-lock.json`: registro exacto de versiones de dependencias.

package.json: dependencias, scripts y configuraciones principales.

readme.md: guía inicial e información general del proyecto.

Etapa 2 — Documentación Técnica

Documentación de Archivos de la API de Conexión Node.js (Base de Datos)

En esta parte se documentaron los archivos encargados de conectar el frontend del sitio web de la Escuela de Arte Martín Malharro con su backend. Para llevar a cabo esta tarea, primero se analizó y generó la estructura de archivos del programa, identificando cuáles corresponden al área de APIs y cuáles a la configuración de Node.js, para luego proceder con la documentación de cada uno.

En términos generales se documentó cómo la aplicación se configura para funcionar, qué librerías utiliza y cómo se inicializa el proyecto. También se explicó cómo los usuarios pueden registrarse e iniciar sesión tanto con credenciales propias como con Google, cómo el sistema reconoce el rol de cada usuario y qué permisos tiene, cómo el sitio obtiene desde la API todo el contenido que muestra en pantalla como textos, imágenes, eventos, documentos y usinas, y cómo se pueden crear y modificar datos desde el frontend.

Respecto a la base de datos: al no existir archivos PostgreSQL directos en el repositorio, se dejó en el `readme.md` una explicación de la funcionalidad de la API activa "Strapi" para que futuros programadores entiendan la forma en que fue diseñado originalmente el proyecto.

Documentación de Componentes React y Next.js

A continuación se detallan los componentes principales documentados durante esta etapa:

`config.js` — Configuración Global

Centraliza las variables de entorno necesarias para el funcionamiento del sistema. Expone cuatro constantes principales: `API_URL` (URL base de la API), `URL` (URL del frontend), `clientIDGoogle` (Client ID de Google OAuth) y `API_TOKEN` (token administrativo de Strapi). Todas se cargan mediante `process.env` con el prefijo `NEXT_PUBLIC_` para ser accesibles desde el cliente.

`layout.js` — Estructura Base

Define la estructura base compartida entre todas las páginas del sitio. Incluye el `Toaster` para notificaciones globales y el `UserMenu` como componente de navegación persistente. Aplica los estilos globales y la fuente del sistema. Se ejecuta en el servidor (`Server Component`) y envuelve a todos los demás componentes hijos.

`page.js` — Página Principal

Es la página principal (`home`) del sitio web. Importa y renderiza todos los componentes de contenido: `Carrusel`, `Texto`, `Acordeón`, `Imagen`, `Agenda`, `Documentos` y `Usina`. Funciona como un contenedor de todos los bloques de contenido institucional de la `home`.

`useUser.js` — Hook de Usuario

Hook personalizado de React que administra el estado del usuario autenticado. Lee el token `JWT` y los datos del usuario desde `localStorage`. Expone un objeto `user` con nombre, email y rol, y una función `logout` para cerrar sesión. Se usa como fuente central del estado de autenticación en toda la aplicación.

auth.js — Control de Autenticación

Módulo que expone las funciones `isAuthenticated()` y `getToken()`. OBSERVACIÓN CRÍTICA: utiliza la clave "token" para leer de `localStorage`, mientras que el resto del sistema almacena el JWT bajo la clave "jwt". Esto provoca que `isAuthenticated()` siempre devuelva false aunque el usuario esté logueado correctamente. Requiere corrección urgente para unificar la clave usada.

iniciarSesion.js — Formulario de Login

Componente de formulario para el inicio de sesión tradicional (email + contraseña). Realiza una petición POST a la API de Strapi con las credenciales del usuario. Si la autenticación es exitosa, guarda el JWT en `localStorage` bajo la clave "jwt" y redirige al usuario.

loginWithGoogle.js — Login con Google OAuth

Implementa la autenticación mediante Google OAuth. Usa la librería `@react-oauth/google`. Al recibir el credential token de Google, lo decodifica con `jwt-decode` para extraer datos del usuario. Luego consulta la API de Strapi para verificar si el usuario ya existe. Almacena el JWT bajo la clave "jwt". NOTA: El `API_TOKEN` queda expuesto en el bundle del cliente.

registrar.js — Registro de Usuarios

Componente de formulario de registro de nuevos usuarios. Realiza una petición POST a `/api/auth/local/register` de Strapi. Si el registro es exitoso, guarda el JWT en `localStorage` y redirige al usuario al home.

checkRole.js — Verificación de Roles

Función utilitaria que verifica el rol del usuario autenticado contra la API de Strapi. Lee el JWT de `localStorage` (clave "jwt"), lo envía como Bearer token en el header `Authorization`, y consulta el endpoint `/api/users/me`. Devuelve el rol del usuario para controlar el acceso a secciones protegidas.

handleSave.js — Control y Validación de Guardado

Función genérica para guardar cambios en la API de Strapi. Recibe como parámetros: el ID del documento a editar, el campo a modificar (`campoAModificar`), el nuevo valor, el endpoint de la API y el JWT del usuario. Realiza una petición PUT con el header `Authorization`. OBSERVACIÓN: siempre devuelve `updated.data.texto` en lugar de `updated.data[campoAModificar]`, lo que genera que el valor devuelto no corresponda al campo editado.

agenda.js — Visualización de Eventos

Componente que obtiene y renderiza la agenda de eventos institucionales desde la API de Strapi. Hace un fetch a `/api/eventos?populate=*`. OBSERVACIÓN: si un evento no tiene imagen asociada (imagen es null), la línea `const imageUrl = imagen.url` falla con `TypeError` y el carrusel completo de Agenda desaparece. Requiere validación con optional chaining: `imagen?.url`.

carrusel.js — Componente de Slider

Componente de carrusel visual que muestra imágenes mediante rotación automática. Usa la librería react-slick para el comportamiento del slider. Obtiene las imágenes desde la API de Strapi.

document.js — Documentos Institucionales

Componente que lista y permite descargar documentos institucionales obtenidos desde la API de Strapi. OBSERVACIÓN: actualmente no valida la respuesta HTTP antes de leer el JSON, lo que puede causar errores silenciosos si la API falla. Se recomienda agregar verificación de `res.ok` antes de llamar a `res.json()`.

userMenu.js — Menú Interactivo del Usuario

Componente de navegación persistente que muestra opciones según el estado de autenticación del usuario. Usa el hook `useUser()` para determinar si hay un usuario logueado y mostrar su nombre, opciones de logout y enlaces al panel de creación de usinas (según el rol).

usina.js — Visualización de Publicaciones

Componente que obtiene y renderiza las publicaciones (usinas) desde la API de Strapi. Hace un fetch a `/api/usinas?populate=*`. Muestra título, descripción e imagen de cada publicación en una grilla o lista.

imagen.js / imagenById.js — Manejo de Imágenes

`imagen.js` obtiene todas las imágenes desde la API. `imagenById.js` busca una imagen específica por su identificador único (`imagenID`). OBSERVACIÓN: el acceso a `data[0].imagen.formats.thumbnail.url` falla con `TypeError` si `data` está vacío o si la imagen no tiene el formato thumbnail generado. Requiere validación con optional chaining.

text.js / textoById.js — Manejo de Textos

`text.js` es un componente client-side que obtiene un texto específico por ID y permite edición en línea si el usuario tiene el rol correspondiente. `textoById.js` es la función asíncrona que realiza el fetch a Strapi filtrando por `textoID`. OBSERVACIÓN: si el `textoID` buscado no existe en la API, `data[0]` es `undefined` y la línea `setID(getData.data[0].documentId)` lanza un `TypeError`.

acordeon.js / acordeonById.js — Contenido Desplegable

`acordeon.js` renderiza un componente de acordeón que muestra contenido desplegable (listas de items). `acordeonById.js` busca el acordeón específico por su identificador. OBSERVACIÓN: `return data[0].textos || null` falla si `data` es un array vacío. Requiere: `return data[0]?.textos ?? null`.

crearUsina.js / usinaProtegida.js — Creación de Contenido

`crearUsina.js` es el formulario para crear nuevas publicaciones (usinas). OBSERVACIÓN CRÍTICA: realiza los fetch a `/upload` y `/usinas` sin incluir el header `Authorization: Bearer ${jwt}`, a diferencia del resto de las operaciones de escritura del proyecto. Si el rol 'Public' de Strapi no tiene permiso de creación, toda creación de usina fallará con error 403, incluso estando logueado. `usinaProtegida.js` es el wrapper que verifica la existencia de un token antes de renderizar el formulario de creación. Solo verifica la existencia del token en `localStorage`, sin validación desde el servidor.

Diagramas de Arquitectura y Flujo

Arquitectura general del sistema:

Componente	Tecnología	Descripción
Frontend	Next.js 15 + React 19	Interfaz de usuario, renderizado SSR/SSG/Client
Backend / API	Strapi (headless CMS)	Repositorio externo en Render.com
Base de datos	PostgreSQL	Almacenamiento persistente de datos
Autenticación	JWT + Google OAuth	Tokens almacenados en localStorage
Gestor de procesos	PM2	Mantiene la app Node.js activa en el servidor
Servidor web	Nginx	Proxy inverso gestionado por CloudPanel
Sistema operativo	Ubuntu 24.04 LTS	Servidor VPS con IP 168.181.184.177

Flujo de datos principal:

- El usuario accede al sistema desde un navegador web.
- Next.js renderiza la interfaz (combinando SSR, SSG y Client Components según la página).
- El frontend realiza solicitudes HTTP a la API de Strapi para obtener contenido (textos, imágenes, agenda, documentos, usinas).
- Strapi consulta PostgreSQL y devuelve los datos en formato JSON.
- Para operaciones autenticadas, el cliente envía el JWT en el header Authorization.
- Google OAuth provee autenticación externa; el token de Google se intercambia por un JWT de Strapi.

Dependencias y Módulos del Sistema

El archivo package.json registra las siguientes dependencias principales:

Paquete	Versión	Función
axios	^1.9.0	Cliente HTTP para peticiones a la API
next	15.5.9	Framework React para SSR/SSG/App Router
react	^19.0.0	Librería de interfaz de usuario
react-dom	^19.0.0	Renderizado DOM de React
react-hot-toast	^2.6.0	Sistema de notificaciones toast
react-icons	^5.5.0	Librería de íconos SVG
react-markdown	^10.1.0	Renderizado de contenido Markdown
react-slick	^0.30.3	Componente de carrusel/slider

slick-carousel	^1.8.1	Estilos base para react-slick
----------------	--------	-------------------------------

Scripts disponibles (definidos en package.json):

Script	Función
npm run dev	Inicia el entorno de desarrollo completo mediante el script scripts/dev.js
npm run dev:frontend	Inicia únicamente el frontend Next.js en el puerto 3000
npm run dev:backend	Inicia únicamente el backend ubicado en la carpeta backend
npm run build	Genera la versión compilada para producción
npm run start	Ejecuta la versión compilada del proyecto
npm run lint	Analiza el código y detecta posibles errores o malas prácticas

Etapa 3 — Tutorial de Deployment

La Etapa 3 corresponde a la validación y verificación del sistema en producción. El Equipo A participó en la verificación de integridad del sistema junto con el Equipo B (tarea compartida).

Tarea 1: Validar Integridad del Sistema

Se verificó que los archivos del sistema estuvieran presentes y correctamente configurados mediante `ls -la`:

```
package.json:      -rw-rw-r-- 1 u4395av_mm u4395av_mm 628 May 13 19:45
package.json
package-lock.json: -rw-rw-r-- 1 u4395av_mm u4395av_mm 89290 May 13 19:46
package-lock.json
.env.local:        -rw-rw---- 1 u4395av_mm u4395av_mm 58 Oct 29 2025
.env.local
.env.production:   -rw-rw---- 1 u4395av_mm u4395av_mm 43 Oct 30 2025
.env.production
node_modules:      drwxrwx--- 133 u4395av_mm u4395av_mm 4096 May 13 19:46
node_modules
src:               drwxrwxr-x 3 u4395av_mm u4395av_mm 4096 Oct 29 2025 src
.next:            drwxrwx--- 7 u4395av_mm u4395av_mm 4096 Oct 31 2025
.next
```

Contenido verificado de `.env.local`:

- PORT
- NEXT_PUBLIC_API_URL

Contenido verificado de `.env.production`:

- NEXT_PUBLIC_API_URL

Variables confirmadas en `./backend/.env`:

```
# Base de datos
PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD, DATABASE_URL
# Servidor
PORT, NODE_ENV, CLIENT_URL
# JWT
JWT_SECRET, JWT_EXPIRES_IN
# Administrador
ADMIN_EMAIL, ADMIN_PASSWORD
# SMTP
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS, EMAIL_FROM
```

Verificación de PostgreSQL:

```
psql --version: psql (PostgreSQL) 16.13 (Ubuntu 16.13-0ubuntu0.24.04.1)
```

Confirmación de que el sitio está levantando: `curl http://localhost:3000`

Tarea 2: Verificar Compilación y Dependencias

Versiones instaladas verificadas:

```
node -v: v18.19.1
npm -v: 9.2.0
```

Dependencias verificadas (`cat package.json`):

```
{
```

```
"name": "my-app",
"version": "0.1.0",
"private": true,
"scripts": {
  "dev": "node scripts/dev.js",
  "dev:frontend": "next dev --turbo pack -p 3000",
  "dev:backend": "npm --prefix backend run dev",
  "build": "next build",
  "start": "next start",
  "lint": "next lint",
  "postinstall": "npm --prefix backend install"
},
"dependencies": {
  "axios": "^1.9.0",
  "next": "15.5.9",
  "react": "^19.0.0",
  "react-dom": "^19.0.0",
  "react-hot-toast": "^2.6.0",
  "react-icons": "^5.5.0",
  "react-markdown": "^10.1.0",
  "react-slick": "^0.30.3",
  "slick-carousel": "^1.8.1"
}
}
```

Verificación del build (ls .next): app-build-manifest.json, BUILD_ID, build-manifest.json, cache, diagnostics, export-marker.json, images-manifest.json, next-minimal-server.js.nft.json, next-server.js.nft.json, package.json, app-path-routes-manifest.json, prerender-manifest.json, react-loadable-manifest.json, required-server-files.json, routes-manifest.json, server, static, trace, types.

Estado de PM2 (pm2 list): Lista vacía, sin procesos activos al momento de la verificación.

Logs de PM2 (pm2 logs --lines 20): Registros de PM2 con notificaciones de versión desactualizada (recomendación de upgrade a 7.0.1), generadas entre el 19 y 26 de mayo de 2026:

```
PM2 | 2026-05-19T14:32:23: PM2 log: Worker Interval : 30000
PM2 | 2026-05-19T14:32:23: PM2 log: Process dump file :
/home/tecnica5/.pm2/dump.pm2
PM2 | 2026-05-19T14:32:23: PM2 log: Concurrent actions : 2
PM2 | 2026-05-19T14:32:23: PM2 log: SIGTERM timeout : 1600
PM2 | 2026-05-19T14:32:23: PM2 log: Runtime Binary : /usr/bin/node
[PM2] This PM2 is not UP TO DATE — Upgrade to version 7.0.1 (repetido desde
19-05 al 26-05-2026)
```

Etapa 4 — Testing y Validación

La Etapa 4 constituye la fase de testing y validación del proyecto institucional de la Escuela de Artes Visuales Martín A. Malharro. Su objetivo principal es verificar que el sistema funcione correctamente en el entorno de producción, detectar errores o inconsistencias en la documentación generada en etapas previas, y confirmar que el tutorial de deployment elaborado por el Equipo B puede ser replicado de manera independiente por cualquier integrante del equipo técnico.

Esta etapa integra el trabajo realizado por ambos equipos: el Equipo A aportó la documentación del frontend, backend y arquitectura, mientras que el Equipo B documentó la infraestructura, el servidor GNU/Linux, las variables de entorno y el proceso de deployment. La validación cruzada de ambas fuentes permite detectar brechas, inconsistencias o procedimientos que requieren ajuste.

Objetivos de la Etapa 4:

- Verificar el funcionamiento general del sistema en producción.
- Detectar errores o inconsistencias en la documentación técnica.
- Validar la replicabilidad del tutorial de deployment elaborado en la Etapa 3.
- Ajustar los procedimientos técnicos según los hallazgos encontrados.

Tarea 1 — Verificación del Funcionamiento General

Se verificó el estado actual del sistema en el servidor de producción (IP: 168.181.184.177) mediante acceso SSH al usuario tecnica5 por el puerto 5174.

1.1 Conectividad y acceso al servidor

```
ssh -p 5174 tecnica5@168.181.184.177
```

Resultado: Conexión SSH establecida correctamente. El servidor responde en el puerto 5174.

1.2 Estado del sistema operativo y recursos

Parámetro	Valor relevado	Estado
Sistema operativo	Ubuntu 24.04.4 LTS (Noble Numbat)	OK
RAM total / en uso	7,8 GB / 1,2 GB (6,6 GB disponibles)	OK
Espacio en disco (/)	39 GB / 14 GB en uso (36%)	OK
Zona horaria	America/Argentina/Buenos_Aires (-03)	OK
Sincronización NTP	Activa — reloj sincronizado	OK

1.3 Estado de servicios activos

Servicio	Puerto	Estado
Nginx (proxy inverso)	80 / 443	Activo
PostgreSQL v16.13	5432	Instalado

SSH	22 / 5174	Activo
CloudPanel (admin)	8443 (HTTPS)	Activo
Fail2ban	—	Activo
Node.js / Next.js (PM2)	3000	⚠ Sin proceso PM2 activo

1.4 Estado de la aplicación Next.js

```
ls -la /home/u4395av_mm/htdocs/avmartinmalharro.edu.ar/
```

Archivo / Directorio	Observación	Estado
package.json	Proyecto my-app v0.1.0 — Next.js 15.5.9	Presente
package-lock.json	89.290 bytes — versiones bloqueadas	Presente
.env.local	Permisos 660 — propietario u4395av_mm	Presente
.env.production	Permisos 660 — propietario u4395av_mm	Presente
node_modules/	Directorio de dependencias instaladas	Presente
.next/	Build de producción generado	Presente
src/	Código fuente del proyecto	Presente
backend/	Directorio backend Node.js	Presente

1.5 Verificación del build

```
ls .next
app-build-manifest.json | BUILD_ID | build-manifest.json | cache |
diagnostics
export-marker.json | images-manifest.json |
next-minimal-server.js.nft.json
next-server.js.nft.json | package.json | prerender-manifest.json
react-loadable-manifest.json | required-server-files.json |
routes-manifest.json
server | static | trace | types
```

Resultado: El build de producción está completo. Todos los archivos de compilación de Next.js están presentes.

1.6 Versiones instaladas

Herramienta	Versión relevada	Mínimo requerido	Estado
Node.js	v18.19.1	v18.18	OK
npm	9.2.0	—	OK
PM2	6.0.13	—	⚠ Desactualizado (disponible 7.0.1)
PostgreSQL	16.13	—	OK
Git	2.43.0	—	OK

Nginx	1.30.0	—	OK
-------	--------	---	----

Tarea 2 — Detección de Errores en la Documentación

Se realizó una revisión cruzada entre la documentación técnica del Equipo A (frontend, backend, arquitectura) y la documentación del Equipo B (infraestructura, deployment, variables de entorno) con el fin de identificar inconsistencias, omisiones o errores que puedan afectar la replicabilidad del sistema.

2.1 Errores detectados en el código fuente (Equipo A)

ID	Archivo	Descripción del error	Impacto
E-01	auth.js	El token se almacena con la clave "token" pero el sistema lo busca como "jwt". Las funciones de autenticación no tienen efecto real.	Alto — bloquea funcionalidad de login
E-02	usinaProtegida.js	La protección del formulario de Usina solo verifica existencia de token en el navegador, sin validación server-side.	Alto — acceso no autorizado posible
E-03	config.js / frontend	El token administrativo de Strapi está expuesto como variable pública, visible desde el navegador.	Crítico — exposición de credenciales
E-04	readme.md (DB)	No existen archivos PostgreSQL propios. La BD es gestionada por Strapi. Se documentó solo la API de Strapi como referencia.	Medio — puede generar confusión

2.2 Hallazgos de seguridad en el servidor (Equipo B)

ID	Elemento	Descripción	Prioridad
S-01	xmrig-6.21.0 / xmrig.tar.gz	Software de minería de criptomonedas. Presencia sospechosa en servidor web. Posible indicador de compromiso (cryptojacking).	URGENTE
S-02	exploited.log (813 B)	Nombre altamente sospechoso. Posible	URGENTE

		indicador de intrusión o explotación del servidor.	
S-03	scanner_linux (0 B)	Ejecutable de escaneo de red. Origen y propósito desconocido. Requiere revisión.	Alta
S-04	data.log (1,73 GB)	Archivo de log de gran tamaño. Ocupa espacio significativo. Requiere rotación o limpieza.	Media
S-05	authorized_keys (vacío)	Autenticación SSH usa contraseña en lugar de claves públicas. Menor seguridad.	Media

2.3 Inconsistencias en la documentación

ID	Sección afectada	Inconsistencia detectada
D-01	Etapa 3 — Variables de entorno	El .env.local del servidor tiene 2 variables. El manual de Etapa 3 indica 4. Hay diferencia entre el entorno real y la guía.
D-02	PM2 — Ecosistema	No existe ecosystem.config.js. El sistema no está configurado con PM2 en forma persistente. La guía de deployment debe incluir este paso.
D-03	PM2 — Lista de procesos	pm2 list retorna tabla vacía. El proceso Next.js no está corriendo bajo PM2. El servidor no reinicia el proceso ante reinicios.
D-04	Backups — Crontab	No hay tareas cron configuradas para tecnica5. Los backups no están automatizados pese a que existe la carpeta ~/backups/databases/.

Tarea 3 — Validación de Replicabilidad del Deployment

Se ejecutó el tutorial de deployment elaborado en la Etapa 3 desde cero, simulando el procedimiento que seguiría un nuevo integrante del equipo técnico. A continuación se documenta el resultado de cada paso.

3.1 Checklist de validación del tutorial

Paso	Acción verificada	Resultado	Observación
------	-------------------	-----------	-------------

1	node -v >= 18.18 instalado	v18.19.1	OK
2	npm -v disponible	9.2.0	OK
3	git --version disponible	2.43.0	OK
4	git clone del repositorio	Exitoso	OK
5	npm install sin errores	1284 packages instalados	OK
6	.env.local creado con variables	⚠ Ver D-01	Ajustar variables según entorno real
7	npm run build exitoso	Compiled successfully	OK
8	curl http://localhost:3000 responde	Sitio accesible	OK
9	pm2 start configurado	⚠ Sin ecosystem.config.js	Ver D-02 — crear archivo
10	pm2 save + startup systemd	⚠ Sin procesos guardados	Ver D-03 — ejecutar después del paso 9

3.2 Procedimiento para completar la configuración de PM2

Durante la validación se detectó que PM2 no tiene procesos activos ni archivo de ecosistema configurado. Para completar el deployment se deben ejecutar los siguientes pasos:

Paso 1 — Crear el archivo ecosystem.config.js en la raíz del proyecto:

```
// ecosystem.config.js
module.exports = {
  apps: [
    {
      name: 'malharro-frontend',
      script: 'node_modules/.bin/next',
      args: 'start',
      cwd: '/home/u4395av_mm/htdocs/avmartinmalharro.edu.ar',
      env: { NODE_ENV: 'production', PORT: 3000 },
      watch: false,
      autorestart: true,
      max_restarts: 10
    },
    {
      name: 'malharro-backend',
      script: 'server.js',
      cwd: '/home/u4395av_mm/htdocs/avmartinmalharro.edu.ar/backend',
      env: { NODE_ENV: 'production' },
      watch: false,
      autorestart: true,
      max_restarts: 10
    }
  ]
}
```

```
}
```

Paso 2 — Iniciar los procesos con PM2:

```
pm2 start ecosystem.config.js
```

Paso 3 — Guardar la lista de procesos para persistencia:

```
pm2 save
```

Paso 4 — Configurar el inicio automático con systemd:

```
sudo env PATH=$PATH:/usr/bin /usr/local/lib/node_modules/pm2/bin/pm2 startup  
systemd -u tecnica5 --hp /home/tecnica5
```

Paso 5 — Verificar que los procesos quedaron activos:

```
pm2 list
```

Resultado esperado: pm2 list muestra malharro-frontend y malharro-backend con status 'online' y 0 restarts.

Tarea 4 — Ajuste de Procedimientos Técnicos

A partir de los errores y hallazgos documentados en las Tareas 2 y 3, se registran a continuación las correcciones y ajustes aplicados o recomendados para cada ítem.

4.1 Ajustes aplicados al tutorial de deployment (Etapa 3)

Ref.	Ajuste realizado	Estado
D-01	Se actualizó la sección de variables de entorno. Se indica que .env.local en producción contiene PORT y NEXT_PUBLIC_API_URL. Las variables adicionales son necesarias solo en entornos de desarrollo local con Google OAuth activo.	Documentado
D-02	Se incorporó al tutorial el paso de creación del archivo ecosystem.config.js (ver sección 3.2). Se incluyó configuración para frontend y backend.	Incorporado
D-03	Se agregó al tutorial el paso de pm2 save y pm2 startup tras iniciar los procesos, garantizando que el servicio se reinicie automáticamente ante reinicios del servidor.	Incorporado
D-04	Se recomienda agregar tarea cron para backup automático de PostgreSQL. Pendiente de implementación en Etapa 5 (Manual de Mantenimiento).	Pendiente Etapa 5

4.2 Recomendaciones de seguridad para el administrador del servidor

ADVERTENCIA: Los ítems S-01, S-02 y S-03 (xmrig, exploited.log, scanner_linux) deben ser comunicados de inmediato al administrador del servidor. El equipo técnico no debe eliminar estos archivos hasta que se realice una auditoría de seguridad.

Ref.	Recomendación	Responsable
S-01	Investigar y eliminar xmrig. Verificar procesos activos con <code>ps aux grep xmrig</code> . Auditar logs del sistema.	Administrador del servidor
S-02	Revisar contenido de exploited.log. Analizar accesos no autorizados. Reportar al proveedor de hosting si se confirma intrusión.	Administrador del servidor
S-03	Verificar scanner_linux con <code>file scanner_linux</code> y <code>strings scanner_linux</code> . Si es malicioso, eliminar y cambiar todas las contraseñas.	Administrador del servidor
S-04	Limpiar data.log (<code>truncate -s 0 data.log</code>) o configurar <code>logrotate</code> para rotación automática.	Equipo B — Etapa 5
S-05	Configurar autenticación SSH por clave pública: generar par de claves con <code>ssh-keygen</code> y agregar la clave pública a <code>~/.ssh/authorized_keys</code> .	Administrador del servidor
E-03	Mover <code>NEXT_PUBLIC_API_TOKEN</code> a variable de entorno privada del backend. Revisión conjunta con Equipo A.	Equipo A + Equipo B

4.3 Checklist final del tutorial (versión corregida)

Se actualizó el checklist del tutorial de deployment de la Etapa 3 para incluir los pasos de PM2 que faltaban:

- Node.js y npm instalados con versión correcta (`node -v >= 18.18`).
- Repositorio clonado correctamente con `git clone`.
- `npm install` ejecutado sin errores y carpeta `node_modules` creada.
- Archivo `.env.local` creado con las variables correspondientes al entorno.
- `npm run build` finaliza con `Compiled successfully`.
- `ecosystem.config.js` creado en la raíz del proyecto.
- `pm2 start ecosystem.config.js` ejecutado — procesos `malharro-frontend` y `malharro-backend` en estado online.
- `pm2 save` ejecutado — lista de procesos guardada.
- `pm2 startup` configurado — inicio automático con `systemd` habilitado.
- `curl http://localhost:3000` confirma que el sitio responde.
- La página muestra todos los contenidos (textos, imágenes, agenda, carrusel, usinas).

Resumen de Resultados — Etapa 4

A continuación se presenta un resumen consolidado del estado del sistema al cierre de la Etapa 4, con indicación del resultado de cada control realizado.

Área	Resultado	Observación
Conectividad SSH	Verificado	Puerto 5174 — acceso correcto
Sistema operativo	Verificado	Ubuntu 24.04.4 LTS — estable
Recursos del servidor	Verificado	Carga baja — sin saturación
Nginx (proxy)	Activo	Puerto 80/443 operativo
Build de Next.js	Completo	Carpeta .next con todos los archivos
Versiones de software	OK / △	PM2 v6 disponible — versión 7 recomendada
PM2 — procesos activos	△ Pendiente	Requiere ecosystem.config.js y pm2 start
Variables de entorno	△ Ajustado	Diferencia entre servidor y guía — ver D-01
Backups automatizados	△ Pendiente	Sin cron — se aborda en Etapa 5
Seguridad — xmrigr	Requiere acción URGENTE	Escalar al administrador del servidor
Seguridad — exploited.log	Requiere acción URGENTE	Escalar al administrador del servidor
Errores de código (E-01/02/03)	△ Detectados	Documentados — corrección a cargo del Equipo A
Replicabilidad del deployment	Validado (parcial)	Pasos 1-8 correctos. Pasos PM2 incorporados al tutorial.

Conclusión: El sistema se encuentra operativo y el build de producción está completo. El tutorial de deployment es replicable una vez incorporados los ajustes de PM2 documentados en esta etapa.

NOTA: Los hallazgos de seguridad (xmrigr, exploited.log, scanner_linux) requieren atención urgente por parte del administrador del servidor. Se recomienda no postergar esta revisión.

Etapa 5 — Manual Funcional

1. Introducción

Este manual funcional tiene como objetivo describir el funcionamiento general del sistema web que se desarrolló para la Escuela de Artes Visuales Martín A. Malharro. El documento reúne toda la información necesaria para comprender la arquitectura, los componentes, el sistema de autenticación y los flujos de datos del sistema.

2. Tecnologías Utilizadas

Tecnología	Función
Next.js	Framework frontend
React	Interfaz de usuario
Node.js	Entorno de ejecución
PostgreSQL	Base de datos
Strapi	CMS y API
PM2	Gestión de procesos
GNU/Linux	Sistema operativo servidor
Google OAuth	Autenticación externa

3. Arquitectura General del Sistema

El sistema utiliza una arquitectura cliente-servidor.

Funcionamiento general:

10. El usuario accede al sistema desde el navegador web.
11. El frontend desarrollado con Next.js renderiza la interfaz.
12. El frontend realiza solicitudes HTTP a la API.
13. Strapi administra el contenido dinámico.
14. PostgreSQL almacena la información persistente.

4. Estructura General del Proyecto

- Docs/: Contiene documentos, informes y archivos técnicos del proyecto.
- Public/: Contiene archivos públicos accesibles desde el navegador.
- Src/: Contiene el código fuente principal del sistema.
- Src/app/: Núcleo principal de la aplicación desarrollado con Next.js.
- Componentes/: Contiene componentes reutilizables en distintas partes del sistema.
- Hooks/: Funciones reutilizadas de lógica personalizada.
- Login/: Módulos relacionados con autenticación de usuarios.
- Validación/: Módulos encargados del control de permisos y validaciones.
- Styles/: Hojas de estilo generales de la aplicación.

5. Página Principal

Archivo principal: Page.js

Funciones:

- Mostrar contenido institucional.
- Centralizar navegación.
- Permitir acceso a las distintas secciones del sistema.

6. Sistema de Autenticación

Archivos relacionados: Auth.js, iniciarSesion.js, registrar.js, loginWithGoogle.js.

Funcionalidades: El sistema permite registro de usuarios, inicio de sesión tradicional, y autenticación mediante Google OAuth.

Funcionamiento:

15. El usuario ingresa credenciales.
16. El sistema valida la información.
17. Se genera un token de autenticación.
18. El usuario accede al contenido protegido.

Observaciones detectadas durante el análisis técnico:

- Inconsistencias entre el almacenamiento de "token" y "jwt": auth.js usa la clave "token" mientras que el resto del sistema usa "jwt". Esto causa que isAuthenticated() siempre devuelva false aunque el login sea exitoso.
- Validaciones incompletas desde el lado del servidor: usinaProtegida.js solo verifica la existencia del token en el cliente, sin validación server-side.
- Exposición pública del token administrativo de Strapi: NEXT_PUBLIC_API_TOKEN queda visible en el bundle de JavaScript descargable por cualquier usuario.

7. Gestión de Usuarios

Archivo: useUser.js

Funciones:

- Administración de sesión.
- Persistencia de usuario.
- Manejo del estado autenticado.

8. Sistema de Roles y Permisos

Archivos: checkRole.js, handleSave.js

Funciones:

- Validación de permisos.
- Verificación de roles.
- Control de acceso.
- Validación de operaciones.

9. Componentes Principales del Sistema

Agenda (agenda.js): Permite visualizar eventos y actividades institucionales.

Carrusel (carrusel.js): Permite mostrar contenido visual dinámico mediante rotación automática.

Imágenes (imagen.js, imagenById.js): Permiten obtener y renderizar imágenes desde la API.

Textos (text.js, textoById.js): Permiten obtener y mostrar contenido textual dinámico.

Documentos (document.js): Permite visualizar documentos institucionales.

Usina (usina.js, crearUsina.js, usinaProtegida.js):

- Visualizar publicaciones.
- Crear contenido dinámico.
- Restringir acceso mediante autenticación.

10. Navegación y Layout

Archivos: Layout.js, userMenu.js

Funciones:

- Mantener estructura visual compartida.
- Administrar navegación.
- Generar menú interactivo de usuario.

11. Configuración General

Archivo: Config.js

Función: Centralizar funciones generales del sistema. Almacenar rutas y variables utilizadas por la aplicación.

12. API y Base de Datos

El sistema utiliza Strapi como API principal para administrar contenido dinámico. La información almacenada en PostgreSQL es consumida desde el frontend mediante solicitudes HTTP.

El sistema obtiene desde la API:

- Textos
- Imágenes
- Agenda
- Documentos
- Publicaciones (Usinas)

NOTA sobre la base de datos: al no existir archivos PostgreSQL directos en el repositorio analizado, se dejó en el readme.md del proyecto una explicación de la funcionalidad de la API activa Strapi, para que futuros programadores entiendan la forma en que fue diseñado originalmente el proyecto y cómo se gestiona la persistencia de datos.

13. Scripts del Sistema

Script	Función
npm run dev	Inicia el entorno de desarrollo completo mediante el script scripts/dev.js.
npm run dev:frontend	Inicia únicamente el frontend Next.js en el puerto 3000.

npm run dev:backend	Inicia únicamente el backend ubicado en la carpeta backend.
npm run build	Genera la versión compilada para producción.
npm run start	Ejecuta la versión compilada del proyecto.
npm run lint	Analiza el código y detecta posibles errores o malas prácticas.

14. Observaciones y Recomendaciones Finales

A partir del análisis técnico completo del sistema, el Equipo A identifica las siguientes observaciones y recomendaciones:

Observaciones críticas:

19. Inconsistencia en la clave JWT (auth.js usa "token", el resto usa "jwt"): provoca que isAuthenticated() y getToken() sean inefectivas. Impacto alto. Solución: unificar a "jwt" en todo el proyecto.
20. Falta el header Authorization en crearUsina.js: los fetch a /upload y /usinas no envían el JWT, causando error 403 al crear contenido. Impacto alto. Solución: agregar Authorization: Bearer \${jwt} en ambas peticiones.
21. Token administrativo de Strapi expuesto públicamente: NEXT_PUBLIC_API_TOKEN es visible en el bundle del navegador. Impacto de seguridad moderado-alto. Solución: limitar permisos del token a solo lectura sobre /users en el panel de Strapi.

Observaciones de calidad:

22. Falta de optional chaining en múltiples componentes: acordeonByID.js, imagenByID.js, agenda.js y text.js acceden a propiedades anidadas sin verificar si el dato existe, causando TypeErrors en producción.
23. handleSave.js devuelve siempre data.texto en lugar de data[campoAModificar]: el valor devuelto no corresponde al campo editado, causando que la UI muestre datos obsoletos inmediatamente después de guardar.
24. document.js no valida res.ok antes de leer el JSON: puede causar errores silenciosos si la API falla.
25. No existe archivo .env.example en el repositorio: dificulta la configuración para nuevos colaboradores.
26. El manejo de autenticación está distribuido entre múltiples módulos sin centralización: auth.js, useUser.js, checkRole.js y componentes individuales leen localStorage directamente, dificultando el mantenimiento.